EL POLIMORFISMO EN LA PROGRAMACIÓN ORIENTADA A OBJETOS

La Programación Orientada a Objetos (POO) se sustenta en principios fundamentales como la abstracción, la encapsulación, la herencia y el polimorfismo. Este último es quizás el que mayor flexibilidad y dinamismo aporta al diseño de software, pues permite que un mismo método, función o referencia adopte múltiples formas y se comporte de manera diferente según el contexto. La palabra polimorfismo proviene del griego polymorphos, que significa "muchas formas", y describe con precisión esta capacidad de adaptación dentro de los sistemas orientados a objetos (Sommerville, 2011).

El polimorfismo se manifiesta, en primer lugar, a través de la asignación polimórfica. Esto sucede cuando una variable declarada como un tipo general, usualmente una clase padre, puede contener referencias a objetos de cualquiera de sus clases hijas. De este modo, aunque el compilador reconozca un tipo estático, el comportamiento real del programa dependerá del tipo dinámico asignado en tiempo de ejecución. Por ejemplo, una variable declarada como Animal puede referirse a un Perro o a un Gato, y aunque ambos comparten la misma interfaz definida en la superclase, la ejecución de un método como hacerSonido() producirá resultados diferentes según el tipo concreto del objeto. Esta característica no solo facilita la reutilización del código, sino que también fomenta la extensibilidad, ya que permite añadir nuevas clases sin alterar la lógica principal del sistema (Gamma et al., 1995).

Es importante diferenciar entre tipos estáticos y tipos dinámicos. El tipo estático corresponde al declarado en el código fuente y es el que utiliza el compilador para verificar la coherencia en las operaciones. El tipo dinámico, por el contrario, es el que adquiere el objeto en tiempo de ejecución y determina qué implementación se ejecutará. Esta dualidad

proporciona seguridad en el proceso de compilación y, al mismo tiempo, flexibilidad en la ejecución. Por ejemplo, aunque el compilador reconozca que una variable es de tipo Figura, en ejecución esta podría ser un Círculo o un Cuadrado, y el método dibujar() se resolverá de acuerdo con la clase concreta (Stroustrup, 2013).

La herencia y los sistemas de tipos constituyen la base sobre la cual se construye el polimorfismo. Gracias a la herencia, las subclases heredan atributos y comportamientos de las superclases, y tienen la capacidad de sobrescribir métodos para ajustarlos a sus propias necesidades. Esta relación jerárquica permite que las clases compartan una estructura común, mientras que cada una conserva la posibilidad de especializar sus comportamientos. En los sistemas de tipos, esta jerarquía asegura que un objeto de una subclase pueda ser tratado como un objeto de su superclase, manteniendo la coherencia en la ejecución y promoviendo la interoperabilidad entre componentes (Booch, 2007).

La compatibilidad de tipos es otro aspecto clave del polimorfismo. Este principio asegura que una subclase pueda sustituir a su superclase sin alterar el correcto funcionamiento del programa, siempre que respete las reglas de comportamiento definidas. Este concepto fue formalizado en el Principio de Sustitución de Liskov, el cual establece que los objetos de una clase base deben poder ser reemplazados por objetos de sus subclases sin modificar las propiedades deseables del sistema. Dicho principio garantiza que las jerarquías de clases no solo compartan estructura, sino también semántica de uso (Liskov & Wing, 1994).

Por otro lado, el polimorfismo también se manifiesta mediante la sobrecarga, una técnica que permite definir varios métodos con el mismo nombre pero con diferentes parámetros. A diferencia de la sobrescritura, que se resuelve en tiempo de ejecución, la sobrecarga se resuelve en tiempo de compilación. Así, un mismo nombre de método puede adaptarse a

distintos contextos de invocación, dependiendo de los argumentos proporcionados. Por ejemplo, un método llamado imprimir puede recibir un entero, una cadena o un objeto, y el compilador seleccionará automáticamente la versión correspondiente según los parámetros. Esta forma de polimorfismo incrementa la legibilidad del código y reduce la duplicación innecesaria.

Referencias

Booch, G. (2007). *Object-Oriented Analysis and Design with Applications*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Liskov, B., & Wing, J. (1994). A behavioral notion of subtyping. *ACM Transactions on Programming Languages and Systems*, 16(6), 1811-1841.

Sommerville, I. (2011). *Software Engineering* (9th ed.). Addison-Wesley.

Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.